

Computational Intelligence

Report Lab2 (CNN)

Author: Jakub Jabłoński

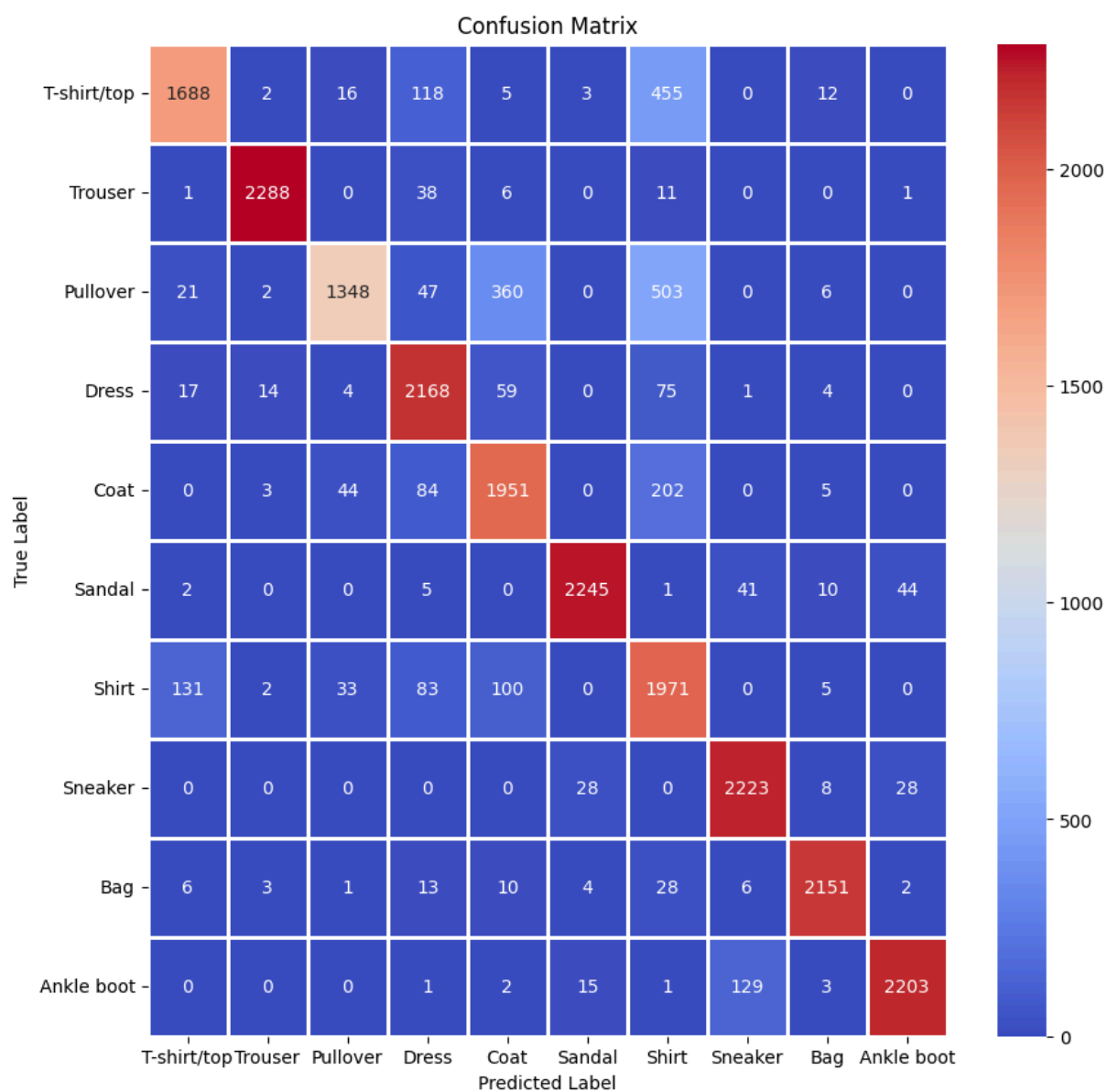
Section 2.1

- a) **Select number of epochs being a tradeoff between processing time and accuracy.**

I've tested a few numbers of epochs and batch sizes, and those values were the best tradeoff between processing time and accuracy:

epochs = 60, batch_size=256

- b) **Analyze the confusion matrices. Indicate which articles were misclassified and why.**



This confusion matrix looks good because it has a lot of matching classes on the diagonal, which means the model is well-trained.

The most misclassified articles are:

- T-shirt/top - This class is confused with the Dress and Shirt classes, which are, in fact, very similar.
- A pullover is confused with a coat and shirt; both have long sleeves
- Coat is confused with a shirt, because of long sleeves
- Shirt - The shirt is compared to T-shirt, Dress, Coat, most likely because of the shape of this article
- Ankle boot - Compared to a Sneaker, both of those articles are shoes, so model may be confused by that

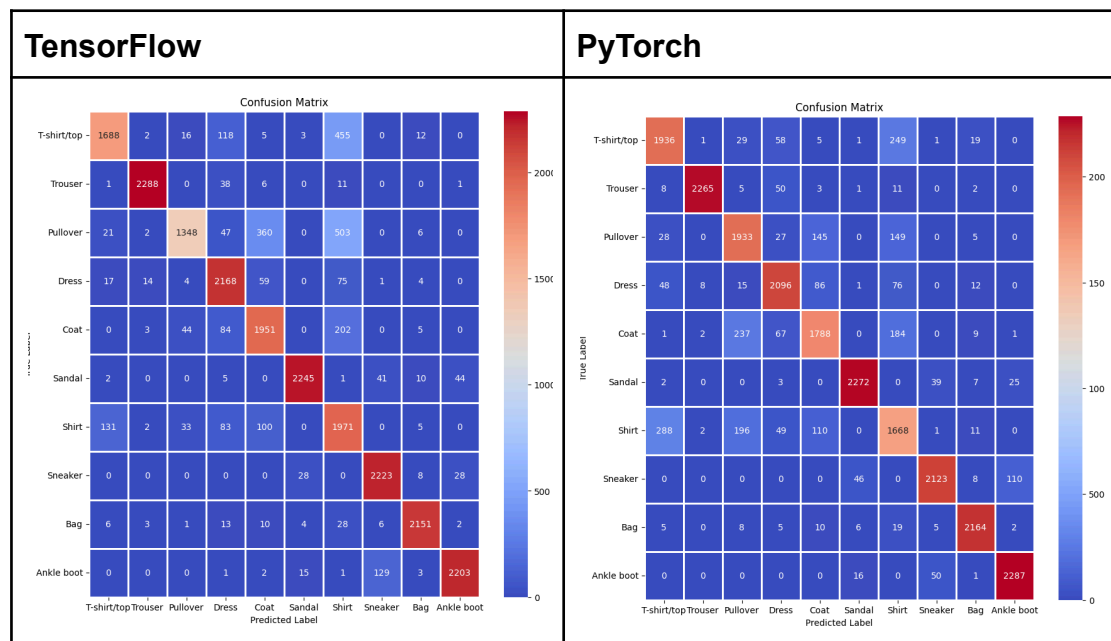
Those misclassifications occur because of the MLP architecture, which takes a flattened vector as an input. It does not take into account the spatial information (every layer is connected with each other). CNN is much better for this task.

c) Compare results for TensorFlow and PyTorch for identical hyperparameters. Are they the same?

The PyTorch model for the identical hyperparameters as the TensorFlow model was trained, has lower accuracy:

- TensorFlow result of the last epoch training:
 - Epoch 60/60, accuracy: 0.9705 - loss: 0.0795
- Pytorch result of the last epoch training:
 - Epoch 60/60, Loss: 1.5390, Accuracy: 92.23%

Confusion Matrices:



Even though, PyTorch model is worse, it has the same problems as the TensorFlow one - it misclassifies identical articles.

When it comes to the learning phase, I can see that the TensorFlow model is trained much faster - it gains accuracy faster than the PyTorch one.

Section 2.2

- a) **Compute numbers of parameters CNN depeneding on input size, filter size and depth. Present results as a table**

$$P = (F_{width} * F_{height} * C + 1) * K ,$$

F_{width}, F_{height} - Kernel size

C - Image channels

K - Number of filters

TensorFlow by default uses padding='valid', so it does not apply any zero padding -> that's why we got a smaller tensor output than the input image.

(img_cols, img_rows)	channels	filters	kernel size	Tensor shape after convolution	Tensor shape after pooling	number of parameters
(10,10)	1	1	(3,3)	(None,8,8,1)	(None, 4,4,1)	10
(10,10)	1	5	(3,3)	(None, 8,8,5)	(None,4,4,5)	50
(10,10)	3	1	(5,5)	(None,6,6,1)	(None,3,3,1)	76
(10,10)	3	5	(3,3)	(None,8,8,5)	(None,4,4,5)	140
(16,16)	3	64	(3,3)	(None, 14,14,64)	(None, 7,7,64)	1792

Section 2.3

- a) Define configurations and perform test. Use various models and hyperparameters. Gather results in a table.

	Model	Epochs	Learning_rate	Random_seed	Accuracy	Precision	\
0	MLP_0	20	0.010	42	0.1000	0.1000	
1	MLP_1	20	0.010	42	0.4680	0.4680	
2	MLP_1_ver2	40	0.010	42	0.6497	0.6497	
3	MLP_2	20	0.010	42	0.1005	0.1005	
4	MLP_2_ver2	40	0.005	42	0.8148	0.8148	
5	CNN_0	20	0.010	42	0.8629	0.8629	
6	CNN_1	20	0.010	42	0.8699	0.8699	
7	CNN_1_ver2	40	0.005	42	0.8714	0.8714	
8	CNN_2	20	0.010	42	0.8346	0.8346	
9	CNN_2_ver2	40	0.010	42	0.8648	0.8648	
10	CNN_2_ver3	40	0.005	42	0.8726	0.8726	
	Recall	F1					
0	0.010000	0.018182					
1	0.472022	0.396580					
2	0.671826	0.603363					
3	0.143333	0.019375					
4	0.813153	0.812767					
5	0.864193	0.861637					
6	0.873331	0.870496					
7	0.873972	0.872163					
8	0.840604	0.836743					
9	0.864924	0.864637					
10	0.873870	0.872131					

Something is wrong with MLP_0 and MLP_2, and I cannot spot the problem. Nevertheless, CNNs have much better results than MLPs.

Section 2.4 (Use Tensorflow or PyTorch)

- a) Experiment with various window sizes, e.g. 6,12, 24
- b) Use different numbers of filters and filter width (kernel sizes: ks)
- c) You may also try to add a Dropout layer for regularization
- d) Present results in a table. Report parameters and obtained scores (MAE, RMSE, R2). Display selected plots.

	Config	Window Size	Out Channels 1	Out Channels 2	Kernel Size 1	Kernel Size 2	Dropout Rate	MAE	RMSE	R2
0	w_6	6	64	32	3	3	0.0	0.013154	0.021613	0.566087
1	w_12	12	64	32	3	3	0.0	0.013873	0.021150	0.544978
2	w_12_ks5	12	64	32	5	5	0.0	0.013654	0.021044	0.570417
3	w_24	24	64	32	3	3	0.0	0.011473	0.014579	0.677510
4	w_12_ks3_32_16	12	32	16	3	3	0.0	0.013809	0.021086	0.559088
5	w_12_ks5_128_64	12	128	64	5	5	0.0	0.014876	0.022325	0.526659
6	w_6_do	6	64	32	3	3	0.2	0.013393	0.021409	0.561150
7	w_12_do	12	64	32	3	3	0.2	0.013603	0.021290	0.565402
8	w_24_do	24	64	32	3	3	0.2	0.011633	0.015038	0.679580

The bigger the window is, the better the results are (MAE, RMSE, R2). The dropout layer helped a little bit in the R2 metric, but the difference is not huge. Increasing the kernel size from 3x3 to 5x5 resulted in slightly better results.

Answer the following questions using 1-2 concise sentences:

- a) In the case of binary classification, how many output neurons and activation functions can be used?

We can use two activation functions:

- **Sigmoid** - 1 output neuron - Probability of one class
 - P1 - probability of class 1
 - $P1 = [0,1]$
 - $P2 = 1 - P1$ - probability of class 2
- **Softmax** - 2 output neurons - Probabilities of two classes:
 - P0 - probability of class 1
 - P1 - probability of class 2
 - $P0 + P1 = 1$

- b) What are the padding and stride parameters used in convolution responsible for?

Padding adds additional zero-valued pixels around the image, so we can have input size == output size (without padding, output shape will be lower than input shape).

Stride specifies how many pixels the filter moves in each step.

- c) What parameters of the convolution network are modified during training?

Values of filters, weights and biases

- d) Why for image processing we used 2D convolution and for time series 1D

Images are 2D signals (an Image is a 2D matrix) and therefore it need 2D convolutional filters so they can filter in two dimensions at the same time.

Time series are 1D signals (1D matrix => vector), so we can use 1D filters.